

**WASTE VISION: A DEEPLARNINGARCHITECTURE FOR
INTELLIGENT AND RELIABLE RECYCLABLE
WASTERECOGNITION**

A Project Report Submitted
in
Partial fulfilment of the Requirements
For the Award of the Degree of
Bachelor of Technology
CSE

Submitted By

Kotte Yashvitha	22541A0548	Pedhagamalla Gayatri	22541A0579
Bommina Lakshmi Prasanna	22541A0505	Mallarapu Narasimharao	22541A05D0
Kolluru Tejamahitha	22541A0541	Syed Muneer Basha	23545A0507

Under the Guidance of
Mr.A.V.Rama Krishna M Tech,(PhD)
Associate Professor



2022-2026

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
SRI SARATHI INSTITUTE OF ENGINEERING & TECHNOLOGY
(Approved by AICTE Affiliated to JNTUK, ISO9001:2008 Certified)

NUZVID – 521201, Eluru District, A.P 2022-2026

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
SRI SARATHI INSTITUTE OF ENGINEERING & TECHNOLOGY

(Approved by AICTE Affiliated to JNTUK, ISO 9001:2008 Certified)

NUZVID–521201, Eluru District. AP

2022-2026



CERTIFICATE

This is to certify that the Project Report entitled “**WASTEVISION:A DEEP LEARNING ARCHITECTURE FOR INTELLIGENT AND RELIABLE RECYCLABLE WASTE RECOGNITION**” submitted by Kotte Yashvitha (22541A0548), Bommina Lakshmi prasanna (22541A0505), Kolluru Tejamahitha (22541A0541) , Pedhagamalla Gayatri (22541A0579), Mallarapu Narasimharao (22541A05D0), Syed Muneer Basha(23545A0507) in partial fulfilment of requirements for the award of the degree of **Bachelor of Technology** in **Computer science engineering** to the Jawaharlal Nehru Technological University, Kakinada is a record of Bonafide work carried out by him/her under our guidance and supervision .

This results embodied in project report haven't been submitted to any other university for the award on any degree or diploma by any student of this college.

Signature of the Internal Guide

Mr.A.V.Rama krishna,M.Tech(PhD)

Associate professor

Signature of HOD

Mr. V. Bharat,M.Tech(PhD)

Head of the CSE Dept.

Signature Of The External Examiner

DECLARATION

We hereby declare that this Project Report entitled“**WASTE VISION:A DEEP LEARNING ARCHITECTURE FOR INTELLIGENT AND RELIABLE RECYCLABLE WASTE RECOGNITION**”to Jawaharlal Nehru Technological University, Kakinada, is a genuine work carried out by us, for the partial fulfilment of the degree of Bachelor of Technology in Computer Science & Engineering during the academic year 2022-2026 under the supervision of our guide, in SRI SARATHI INSTITUTE OFENGINEERING AND TECHNOLOGY,NUZVID.

Bommina Lakshmi Prasanna	(22541A0505),
Kolluru Teja mahitha	(22541A0541),
Kotte Yashvitha	(22541A0548),
Pedhagamalla Gayatri	(22541A0579),
Mallarapu Narasimharao	(22541A05D0),
Syed Muneer Basha	(23545A0507).

ACKNOWLEDGEMENT

We thank our honourable Chairman **Sri. S.V. SUDHEER**, for his moral support and the excellent facilities provided. We are grateful to **Prof. S. SRIDHAR, Principal**, Sri Sarathi Institute of Engineering & Technology, Nuzvid, for his support throughout the project for given the permission to undergo the project work.

We would also like to thank Sri Mr. **V. Bharat, M.Tech (PhD) Head of the Department of Computer Science Engineering and Technology** for given us the opportunity to complete the project successfully.

We sincerely thanks to **Mr.A.V.RamaKrishna, M.Tech (PhD) Associate professor** Department of Computer Science and Engineering, Sri Sarathi Institute of Engineering and Technology, Nuzvid for his valuable guidance and support.

We would like to thank all the people who have helped us in completion of project, credit goes to all my team members and our friends those who shows constant encouragement till the completion of this interesting project within the limited time.

I thank for timely suggestions and constant encouragement that boosted up our moral and let to the accomplishment of this project. Firstly, we indebted to my parents, friends and relatives for the physical, psychological and moral extended by them at all time.

We make this project not only for marks but also to increase our knowledge.

Bommina Lakshmi Prasanna (22541A0505)

Kolluru Tejamahitha (22541A0541)

KotteYashvitha (22541A0548)

Pedhagamalla Gayatri (22541A0579)

MallarapuNarasimharao (22541A05D0)

SyedMuneer Basha (23545A0507)

ABSTRACT

Effective waste management is a growing global challenge, requiring intelligent systems that can automatically identify recyclable materials to support sustainable environmental practices. This project, titled “Recyclable Waste Classification,” presents a hybrid deep learning–based web application capable of classifying uploaded waste images into predefined recyclable categories. The system integrates a MobileNetV2 convolutional neural network as a feature extractor and a Support Vector Machine (SVM) classifier for final category prediction. MobileNetV2 efficiently extracts high-level visual features from waste images, while the SVM model ensures robust classification even with limited training data. The uploaded images undergo preprocessing, resizing, and feature extraction before being passed to the trained SVM model. The implementation is deployed using the Flask web framework, providing a user-friendly interface for image upload, prediction display, and user authentication through secure login and registration. A MySQL database is used to manage registered users securely with hashed passwords. The system delivers fast and accurate predictions, demonstrating the capability of lightweight deep learning models to operate effectively on real-world waste classification tasks. The developed application can significantly assist waste segregation processes, improve recycling efficiency, and promote environmentally responsible behavior in communities, industries, and smart city ecosystems.

INDEX

S.No	Title	Page No
1	Chapter 1: Introduction	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Research Motivation	2
1.4	Applications	2
2	Chapter 2: Literature Survey	3
2.1	Deep Learning for Waste Classification	3
2.2	Lightweight CNN Architectures (MobileNetV2)	3
2.3	SVM for Image-Based Classification	4
2.4	Hybrid CNN–SVM Models	4
2.5	Waste Classification Datasets & Challenges	4
2.6	Image Preprocessing Techniques	5
2.7	Automated Waste Segregation Systems	5
2.8	Web-Based Classification Applications	5
2.9	Environmental Impact of AI-Based Recycling	6
2.10	Future Trends in Smart Waste Management	6
2.11	Transfer Learning in Waste Classification	6
2.12	Traditional ML Techniques	7
2.13	Real-Time Waste Sorting Robots	7
2.14	IoT-Enabled Waste Monitoring Systems	7
2.15	Image Segmentation for Waste Detection	8

S.No	Title	Page No
3	Chapter 3: Existing System	9
3.1	Existing System	9
3.2	Limitations	9
4	Chapter 4: Proposed System	10
4.1	Proposed System	10
4.2	Advantages	10
4.3	Algorithms (CNN & SVM)	11
4.3.1	CNN (MobileNetV2)	11
4.3.2	SVM Workflow	12
4.4	Modules Involved	13
5	Chapter 5: System Requirements	15
5.1	Hardware Requirements	15
5.2	Software Requirements	16
5.3	UML Diagrams	17
5.4	Use Case Diagram	18
5.5	Class Diagram	18
5.6	Sequence Diagram	19
5.7	Activity Diagram	19
5.8	ER Diagram	20
5.9	System Environment	20
6	Machine Learning Concepts	24
6.1	Introduction to Machine Learning	24

S.No	Title	Page No
6.2	Types of Machine Learning	24
6.3	Applications of Machine Learning	24
6.4	Challenges in Machine Learning	25
6.5	Learning Roadmap	30
7	Results / Output Screens	48
8	Conclusion	49
9	References	50-52

LIST OF FIGURES

Fig No	Figure Name	Page No
Fig 1.1	Waste Classification System Overview	2
Fig 2.1	MobileNetV2 Architecture	4
Fig 2.2	CNN–SVM Hybrid Model Architecture	5
Fig 3.1	Existing Waste Segregation Process	10
Fig 4.1	Proposed System Architecture	11
Fig 4.2	CNN (MobileNetV2) Feature Extraction Process	12
Fig 4.3	SVM Classification Workflow	13
Fig 4.4	System Module Interaction Diagram	14
Fig 7.1	Use Case Diagram	19
Fig 7.2	Class Diagram	19
Fig 7.3	Sequence Diagram	20
Fig 7.4	Activity Diagram	20
Fig 7.5	ER Diagram	21

CHAPTER–1

INTRODUCTION

1.1 Introduction

Waste generation has increased rapidly in recent years due to population growth, urbanization, and industrial expansion. Proper waste segregation is essential for reducing environmental pollution, conserving natural resources, and enabling efficient recycling processes. However, manual waste sorting is often time-consuming, inconsistent, and highly dependent on human knowledge and accuracy. Many recyclable materials—such as plastics, metals, cardboard, and paper—share similar visual appearances, making correct identification challenging in real-world settings. With advancements in artificial intelligence, computer vision models have shown great potential in automating image-based classification tasks.

This project presents a **Recyclable Waste Classification System** that uses a hybrid deep learning architecture combining **MobileNetV2** for feature extraction and a **Support Vector Machine (SVM)** classifier for final prediction. MobileNetV2 is a lightweight convolutional neural network capable of extracting rich visual features from images.. The system is implemented using a Flask-based web interface, allowing users to upload images, authenticate via secure login, and obtain classification results in real time. By integrating deep learning with a user-friendly interface, this project aims to improve the accuracy and efficiency of recyclable waste identification, supporting sustainable waste-management practices.

1.2 Problem Statement

The traditional waste segregation process relies heavily on manual labor, making it slow, error-prone, and inconsistent. Human operators may find it difficult to classify recyclable materials accurately due to visual similarities, varying shapes, contamination, and lack of awareness. Incorrect waste segregation leads to decreased recycling quality, higher treatment costs, environmental pollution, and increased landfill burden. Furthermore, most existing waste-sorting solutions are either too resource-intensive or lack intelligent automation capabilities. Therefore, there is a critical need for an **automated, intelligent, and accurate image-based waste classification system** that can assist users, industries, and municipalities in identifying recyclable materials.

1.3 Research Motivation

The motivation for this research stems from the global push toward sustainable living and environmentally responsible waste-management practices. As recycling continues to play a vital role in reducing carbon footprint and conserving natural resources, the efficiency of waste sorting becomes a significant bottleneck. Many developing regions still depend on manual waste sorting, which is laborious, unhygienic, and inefficient. This highlights the urgent need for affordable and intelligent solutions that can streamline the recycling pipeline. Additionally, the rise of deep learning and edge AI has made it possible to deploy lightweight yet powerful models like MobileNetV2 on accessible platforms, making automated classification feasible even on low-cost hardware. SVM classifiers further enhance prediction accuracy when combined with deep visual features, making the hybrid approach highly practical. This research is motivated by the desire to create a solution that is not only technically effective but also socially impactful—empowering smart cities, educational programs, households, and industries to adopt more sustainable and intelligent waste disposal practices. Ultimately, the project aims to bridge the gap between modern AI advancements and real-world environmental needs.

1.4 Applications

1. Smart Recycling Bins

Integration with intelligent bins enables automatic detection and sorting of recyclable waste without human involvement.

2. Recycling Plants and Sorting

Facilities Automated waste classification on conveyor belts increases sorting accuracy, reduces contamination, and improves material recovery rates.

3. Smart City Waste-Management Systems

Municipal corporations can deploy this system with IoT-enabled waste infrastructure for smarter segregation and monitoring.

4. Educational Institutions and Awareness Programs

Schools and colleges can use the system as a learning tool for promoting environmental awareness and responsible recycling habit.

CHAPTETR – 2

LITERATURE SURVEY

2.1 Deep Learning for Waste Classification

Several studies have explored the use of deep learning architectures for automating waste segregation due to their superior ability to identify visual patterns in images. Convolutional Neural Networks (CNNs) such as VGG16, ResNet, and Inception have demonstrated high accuracy in classifying recyclable materials like plastic, paper, metal, and glass. Researchers found that CNNs outperform traditional image-processing techniques because of their ability to learn complex, hierarchical features automatically. These models showed strong generalization when trained on large-scale waste datasets, proving their suitability for environmental applications. However, many high-performance CNN models suffer from heavy computational requirements, limiting their deployment on real-time systems or low-powered devices. This limitation motivated the need for lightweight architectures, such as MobileNet, which are more suitable for practical implementations like smart waste bins and mobile applications.

2.2 Lightweight CNN Architectures (MobileNet)

MobileNet and its improved versions (MobileNetV2 and MobileNetV3) have been widely studied for image classification tasks, particularly in scenarios requiring high accuracy with low computational cost. Research shows that MobileNet's depthwise separable convolutions significantly reduce model size while maintaining performance comparable to larger CNNs. Studies in environmental monitoring and waste classification highlight MobileNetV2 as a suitable feature extractor for embedded systems due to its fast inference time and robustness against variations in lighting, orientation, and object shape. Its transfer-learning approach, using pretrained ImageNet weights, further improves classification accuracy even with small datasets. These characteristics make MobileNetV2 an ideal choice for recyclable waste classification tasks, especially for web-based and mobile deployments.

2.3 SVM for Image-Based Classification

Support Vector Machines (SVMs) have been extensively used in computer vision research due to their effectiveness in handling high-dimensional feature spaces. Studies combining CNN feature extraction with SVM classifiers consistently report improved accuracy compared to using softmax classifiers alone. SVM's margin-based decision boundaries make it particularly effective for small and medium-sized datasets, reducing risks of overfitting. In waste classification research, SVM has shown the ability to distinguish between visually similar categories such as paper and cardboard. Researchers emphasize that hybrid models (CNN + SVM) often outperform standalone CNN models by leveraging CNNs for feature learning and SVM for final decision-making. This hybrid method forms the core of many modern image-classification systems.

2.4 Hybrid CNN–SVM Models

Multiple studies indicate that combining CNN feature extraction with SVM classification significantly enhances performance in applications where the dataset size is limited or imbalanced. Researchers found that CNNs provide discriminative features, while SVM's kernel-based approach strengthens classification boundaries. Hybrid models have been tested on various datasets such as TrashNet, TACO (Trash Annotations in Context), and synthetic waste datasets, achieving accuracy improvements of 5–10% compared to end-to-end deep networks. This approach is particularly effective in recycling applications because recyclable waste categories often exhibit subtle differences, making classification challenging. The hybrid architecture helps address these issues by leveraging the strengths of both methods.

2.5 Waste Classification Datasets and Challenges

Studies on waste classification highlight the importance of high-quality datasets to train reliable models. The most common dataset, TrashNet, includes images of paper, plastic, metal, cardboard, and glass, but researchers note that dataset limitations such as poor lighting, cluttered backgrounds, and limited category diversity reduce model performance. Other studies propose using augmented datasets or real-world surveillance images to improve robustness. Researchers emphasize that

waste items often appear deformed, crumpled, or partially hidden, posing additional challenges for image-based classification models.

2.6 Image Preprocessing Techniques

Preprocessing plays a crucial role in improving classification accuracy in waste detection research. Studies show that resizing, normalization, contrast adjustment, and background removal enhance the visibility of relevant features in images. Researchers using MobileNet and SVM architectures commonly employ standardized preprocessing pipelines compatible with ImageNet-trained models, such as scaling pixel values and converting images to a uniform size. These preprocessing steps help models handle variations in camera quality, lighting conditions, and object positioning. Effective preprocessing significantly improves model reliability in real-world waste-management scenarios.

2.7 Automated Waste Segregation Systems

Automated waste segregation systems have gained attention as a solution to inefficiencies in manual sorting. Several studies implement robotic arms equipped with vision-based classification models to sort items on conveyor belts. These systems use cameras, deep learning algorithms, and electromechanical components to identify and separate recyclable waste from general waste streams. Research findings show that integrating AI-based classification into waste-management infrastructure reduces contamination in recycling streams, increases processing speed, and improves worker safety. Such studies highlight the importance of accurate image classification as the foundational step in automated segregation.

2.8 Web-Based Classification Applications

Recent research has explored the integration of machine learning models into web applications for accessibility and ease of use. Studies demonstrate that lightweight CNN models deployed through Flask, Django, or cloud APIs enable real-time image classification without requiring local installations. Web-based systems have been used for agriculture, medical imaging, and environmental monitoring. In waste classification, researchers developed browser-based tools that allow users to upload images and receive instant predictions. These systems improve public awareness of recycling practices and are practical for schools, communities, and smart-city initiatives.

2.9 Environmental Impact of AI-Based Recycling

Research highlights the significant environmental benefits achievable through AI-assisted waste segregation. Automated classification reduces the contamination of recyclables, increases the recovery of valuable materials, and supports a circular economy. Studies emphasize that even small improvements in classification accuracy can lead to substantial long-term environmental gains. AI-based systems also reduce manual labor exposure to hazardous waste, contributing to a safer working environment. These findings underline the societal importance of modern waste-classification technologies.

2.10 Future Trends in Smart Waste Management

Recent literature suggests a growing shift toward integrating IoT, deep learning, and edge AI for smart waste-management solutions. Researchers predict that smart bins equipped with sensors and AI models will soon become common in urban areas. Emerging technologies such as federated learning, real-time video classification, and automated robotic sorting systems will further enhance waste-management efficiency. Studies also highlight potential for combining image classification with chemical sensing for hybrid waste analysis. These developments demonstrate that AI-driven waste classification will continue to evolve, becoming more accurate, autonomous, and scalable.

2.11 Transfer Learning in Waste Classification

Transfer learning has become a widely adopted strategy in computer vision tasks, especially when working with small or moderately sized datasets. Researchers have shown that models pretrained on large datasets such as ImageNet significantly improve feature extraction for waste images. Studies demonstrate that transfer learning prevents overfitting, reduces computational cost, and speeds up model convergence. In recyclable waste classification, models like MobileNet, Inception, and EfficientNet are frequently fine-tuned to adapt to waste material characteristics. Transfer learning also eliminates the need for collecting thousands of labeled images, making it ideal for real-world projects. This approach aligns with your system's use of MobilNetV2 as a feature extractor, ensuring accuracy even with limited data.

2.12 Waste Detection Using Traditional ML Techniques

Before deep learning became dominant, several studies explored waste classification using traditional machine learning techniques such as Random Forest, KNN, Decision Trees, and SVM. These models required handcrafted features such as texture descriptors (LBP), color histograms, and shape-based metrics. Although these approaches achieved moderate accuracy, they suffered from poor generalization and required extensive feature engineering. However, studies show that combining CNN features with classical ML methods like SVM (as used in your project) produces much better results than using hand-engineered features alone. This validates the hybrid approach implemented in your system.

2.13 Real-Time Waste Sorting Robots

Research on smart waste-sorting robots integrates mechanical actuators, conveyor systems, and computer vision models. These studies show that CNN-driven robotic arms can automatically detect, classify, and pick recyclable objects from mixed waste streams. Researchers achieved high sorting accuracy with image-based classification pipelines, demonstrating the real-world viability of AI-powered waste management. Although robots require additional hardware engineering, the underlying classification model is similar to your CNN + SVM approach. These findings emphasize that accurate waste-classification models are the foundation for automated physical sorting systems.

2.14 IoT-Enabled Waste Monitoring Systems

Studies introduce IoT-enabled waste bins equipped with sensors, cameras, and communication modules to help monitor garbage levels and classify items. Deep learning models embedded in microcontrollers or Raspberry Pi devices classify waste images on the edge. Researchers found that lightweight models like MobileNet and SqueezeNet are ideal for embedded deployment. Your project follows a similar lightweight design philosophy by using MobileNetV2. The literature confirms that such architectures balance accuracy and efficiency, making them suitable for real-world waste management applications.

2.15 Image Segmentation for Waste Detection

Some studies focus not just on classification but also on segmentation of waste objects in cluttered images. Using models like U-Net, Mask R-CNN, and DeepLab, researchers isolate recyclable objects from background noise.

CHAPTER-3

EXISTING SYSTEM

3.1 Existing System

In the existing waste-segregation ecosystem, most recycling processes rely heavily on manual sorting performed by workers or household users. Waste items are visually inspected and separated into categories such as plastic, paper, metal, and glass. This manual approach is slow, inconsistent, and prone to frequent human errors due to the visual similarity between recyclable materials. In many developing regions, the absence of structured recycling systems further worsens the quality of segregation, resulting in contaminated recyclable waste that cannot be reused efficiently. Even when some organizations use basic image-based systems, these tools often depend on traditional computer vision methods such as color segmentation, handcrafted texture features, or simple statistical models. These conventional techniques lack robustness against variations in lighting, object orientation, image noise, and background clutter, making them unreliable for real-world classification. As a result, recycling plants continue to face inefficiencies, increased costs, and reduced material recovery rates. The limitations of existing systems highlight the need for intelligent, automated, and accurate waste-classification solutions.

3.2 Limitations

- **High Dependency on Manual Sorting:** Requires human labor, which is slow, inconsistent, and prone to misclassification.
- **Low Accuracy with Traditional Vision Methods:** Basic image-processing techniques fail under real-world conditions such as poor lighting and varied backgrounds.
- **Inefficient Waste Stream Processing:** Incorrect segregation leads to contamination of recyclable materials and reduces recycling efficiency.
- **Lack of Automation and Scalability:** Most systems cannot operate continuously or handle large volumes of waste.

CHAPTER – 4

PROPOSED SYSTEM

4.1 Proposed System

The proposed system introduces an **AI-powered Recyclable Waste Classification System** built using a hybrid deep learning approach. The system uses **MobileNetV2** as a lightweight feature extractor to capture deep visual patterns from waste images, followed by a **Support Vector Machine (SVM)** classifier for accurate category prediction. This hybrid approach ensures high accuracy even with moderate training datasets. The system is deployed through a user-friendly **Flask web application**, allowing users to upload images and instantly receive classification results. Secure login and registration functionalities are provided using a MySQL backend with password hashing, ensuring safe access control. The model automatically preprocesses images, extracts features, and predicts the waste category in real time. Unlike traditional systems, the proposed solution is capable of handling variations in object size, shape, and lighting because of the robustness of deep-learning features. This automated, scalable, and efficient system is designed to improve recycling accuracy, reduce manual workload, and support the development of smart waste-management infrastructures.

4.2 Advantages

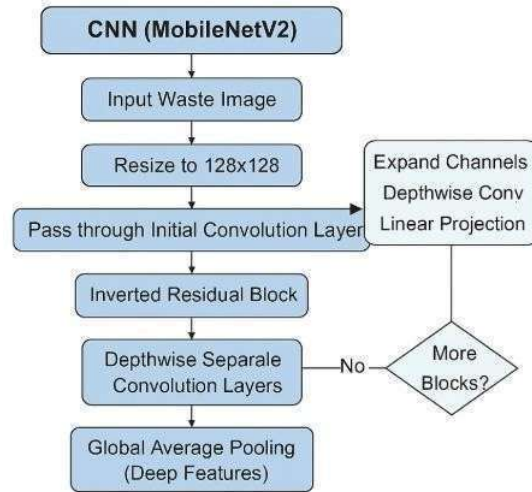
- **High Classification Accuracy:** Deep learning (MobileNetV2) + SVM hybrid architecture improves prediction reliability.
- **Automation of Waste Segregation:** Reduces human labor and speeds up recycling processes.
- **Lightweight and Fast:** MobileNetV2 enables real-time predictions even on limited hardware.
- **User-Friendly Web Interface:** Flask-based platform offers easy image uploads, login, and instant results.

- **Scalable and Extensible:** Can be integrated with IoT smart bins, mobile apps, or recycling plant systems.
- **Improves Recycling Efficiency:** Reduces contamination in recyclable streams, leading to better resource recovery.

4.3 ALGORITHMS:

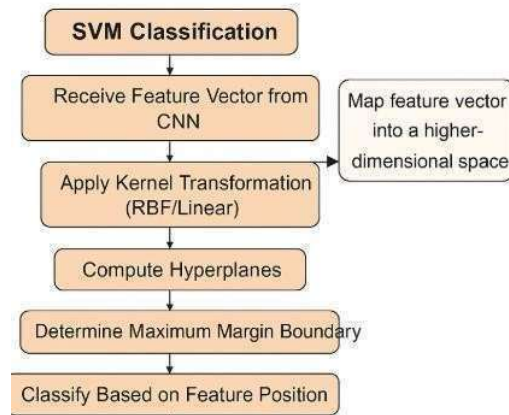
1. CNN (MobileNetV2)

The Convolutional Neural Network (CNN) used in this project is based on the MobileNetV2 architecture, which serves as a powerful feature extractor for waste classification. The workflow begins with the uploaded waste image being resized and normalized to match the expected input format of the CNN. MobileNetV2 first processes the image through an initial convolution layer that captures basic visual patterns such as edges, corners, and color gradients. It then passes the data through a sequence of **inverted residual bottleneck blocks**, which are specially designed to reduce computational complexity while retaining rich feature representations. Each block consists of three main steps: expansion of input channels, lightweight depthwise separable convolution, and linear projection back to a lower-dimensional space. These blocks help the model preserve important spatial relationships, making it effective at identifying textures and shapes commonly found in recyclables such as plastic, paper, metal, and cardboard. After multiple stages of such transformations, the feature map is compressed using **global average pooling**, producing a fixed-length feature vector. This vector contains all the high-level semantic information needed to identify the waste type. Rather than performing classification internally, MobileNetV2 outputs this feature vector to the next stage—SVM—making the CNN a highly efficient deep feature extraction engine within the workflow.



2. SVM Workflow

The Support Vector Machine (SVM) serves as the classification engine that uses the CNN-extracted feature vector to determine the final waste category. Once the feature vector is received from MobileNetV2, the SVM maps it into a higher-dimensional space using kernel functions such as the Radial Basis Function (RBF). This transformation helps the model separate complex, overlapping patterns that commonly occur in waste images—for example, distinguishing between white plastic and white paper, or shiny metal and glossy cardboard. The SVM then constructs an optimal decision boundary, known as a **hyperplane**, which maximizes the margin between different recyclable classes. During prediction, the SVM evaluates the position of the input feature vector relative to these learned hyperplanes and assigns it to the appropriate category. The final numerical output from the SVM is passed through a label encoder to convert it into a human-readable label such as “Plastic,” “Metal,” “Paper,” or “Cardboard.” This two-stage workflow—deep feature extraction using CNN and high-precision classification using SVM—ensures that the system delivers robust and accurate predictions even with limited or mixed training data. The combined architecture allows the model to capture complex visual patterns while maintaining strong generalization performance.



4.4 Modules Involved

1. User Authentication Module

This module handles secure user access to the system. It includes functionalities for user **registration**, **login**, and **logout** using hashed passwords stored in the MySQL database. When a user attempts to log in, the system validates the credentials and starts a session for authenticated access.

2. Image Upload Module

The Image Upload module allows authenticated users to upload images of waste items through the Flask interface. It validates file formats, stores them securely in the “static/uploads” directory, and assigns unique filenames using the `secure_filename` function. This module also ensures that incorrect or unsupported file uploads are prevented, maintaining consistency in the data being processed for classification.

3. Image Preprocessing Module

Once an image is uploaded, the preprocessing module prepares it for feature extraction. It loads the image using OpenCV, resizes it to **128 × 128 pixels**, expands dimensions for model compatibility, and applies MobileNetV2’s `preprocess_input()` function. This ensures uniformity in image size and scaling, allowing the CNN model to extract features accurately regardless of image quality or lighting variations. This module improves robustness and reduces noise during classification.

4. CNN Feature Extraction Module

This module uses **MobileNetV2** (pretrained on ImageNet) as a deep feature extractor. Instead of using MobileNetV2 for classification, the model is configured

with global average pooling to extract meaningful high-level features from the preprocessed image. These features represent texture, patterns, shapes, and visual characteristics necessary for accurate waste classification. This module significantly enhances the learning capability of the SVM model by providing rich, discriminative feature vectors.

5. SVM Classification Module

After feature extraction, the SVM Classification module uses the trained Support Vector Machine (SVM) model to classify the waste item into categories such as plastic, metal, paper, or cardboard. The module loads the pretrained SVM model and label encoder to convert numerical predictions into human-readable labels. This hybrid CNN + SVM design allows the system to achieve high accuracy even with smaller datasets and ensures robust performance across diverse waste images.

6. Result Generation & Display Module

This module receives the predicted class label and returns it to the web interface, along with the uploaded image for visual confirmation. The prediction results, such as “Plastic,” “Metal,” or “Paper,” are displayed clearly on the results page. The module ensures an interactive user experience by providing immediate feedback, making the system easy to use even for non-technical users.

7. Database Management Module

The database module uses MySQL to store user credentials securely. It manages user registration, login verification, and session-based access control. Although the current version primarily stores user data, this module can be expanded to store classification history, image records, or model logs in future enhancements. It ensures seamless communication between the application and the database.

8. File Handling & Storage Module

This module ensures proper management of uploaded files and model files. All uploaded images are saved in a structured directory, and trained models (CNN feature extractor, SVM classifier, and label encoder) are stored in .pkl format for quick loading. This module ensures reliability, prevents overwriting, and supports easy retrieval of files during predictions.

CHAPTER– 5

SYSTEM REQUIREMENTS

5.1 Hardware Requirements

To implement the proposed **Federated Learning-Based Smart Grid Security System**, a standard computing environment is essential to support model training, data processing, and interface execution. The following hardware components are required:

Component	Specification	Description
RAM	Minimum 4 GB	The system requires at least 4 GB of RAM to efficiently handle data preprocessing, model execution, and simultaneous tasks without lag. Higher memory capacity (8 GB or more) is recommended for faster computation and smoother federated learning operations.
Keyboard	Standard Windows Keyboard	A standard keyboard is required for input operations such as command entry, coding, and text data insertion during testing or report
Component	Specification	Description
		generation.
Mouse	Two or Three Button Mouse	Used for graphical user interface navigation, data selection, and program control, enabling easy interaction with web-based dashboards and control panels.
Monitor	SVGA (Super Video Graphics Array)	The system interface, monitoring dashboard, and real-time visualizations are best viewed on an SVGA monitor, which provides high-quality display output for data charts and security alerts.

5.2 Software Requirements

The software setup is essential for developing, executing, and maintaining the proposed system. It includes the operating platform, programming languages, and development frameworks used in implementation.

Component	Specification	Description
Operating System	Windows 10 / 11	The Windows environment offers robust compatibility for Python-based applications, user-friendly GUI support, and an efficient file handling system for project deployment.
Coding Language	Python 3.10+	Python serves as the core programming language due to its extensive libraries for machine learning (Tensor Flow, PyTorch), federated learning (Flower, FedML), and data analysis (NumPy, Pandas).
Back-End	Python	The backend logic is handled by Python scripts managing data flow, model training, aggregation, and security detection mechanisms. It also connects the front-end with the database.
Front-End	HTML, CSS	HTML and CSS are used to design the user interface of the system, providing visualization
Component	Specification	Description
		panels, report dashboards, and control modules for easy interaction.
Designing Tools	HTML, CSS	The graphical layout, page design, and visualization structures are created using standard web technologies, ensuring responsiveness and accessibility across devices.

5.3 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

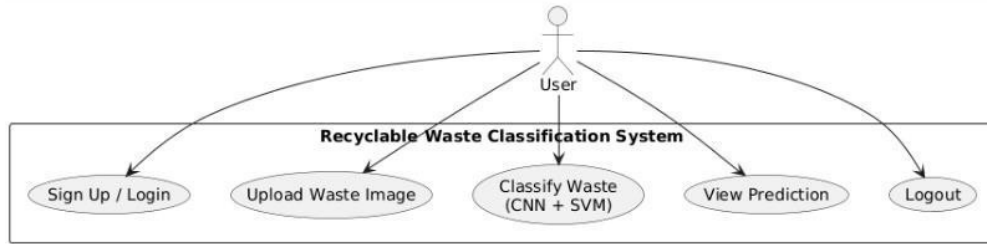
The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

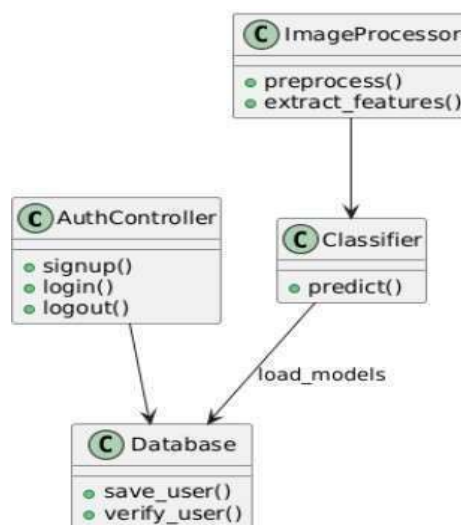
5.4 Use case diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



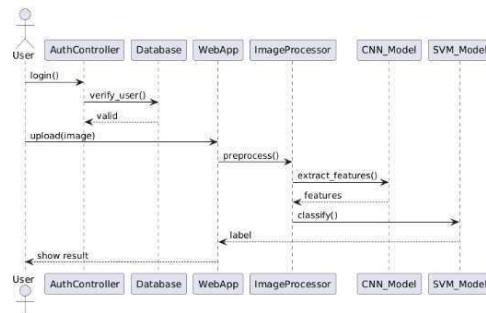
5.5 CLASS DIAGRAM

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram was capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.



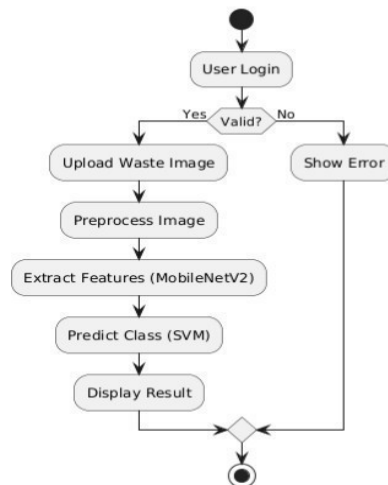
5.6 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows, as parallel vertical lines (“lifelines”), different processes or objects that live simultaneously, and as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

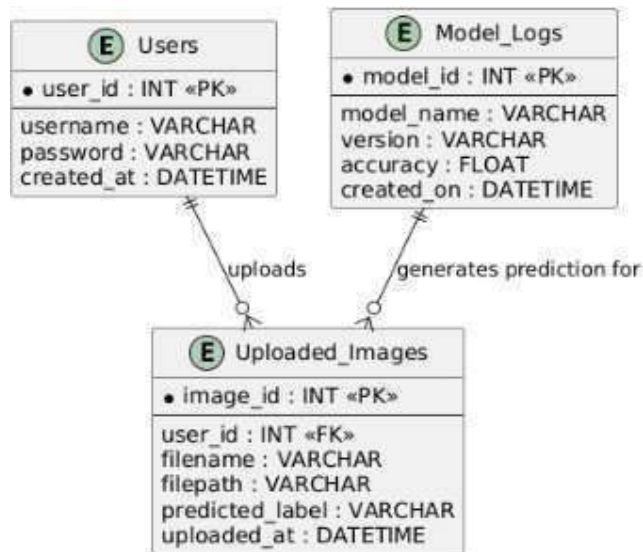


5.7 Activity Diagram

Activity diagrams are graphical representations of Workflows of stepwise activities and actions with support for choice, iteration, and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



5.8 ER Diagram



5.9 System Environment:

What is Python :-

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following .

- [Machine Learning](#)
- GUI Applications (like Kivy, Tkinter, PyQtetc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it *contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more.* So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and [machine learning](#), automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since [Python](#) is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a

requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonnelle**.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java DataBase Connectivity)** and **ODBC (Open DataBase Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python :-

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde&Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill

Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI).

5.10 Introduction to Machine Learning

Machine Learning (ML) is a branch of artificial intelligence (AI) that enables computers to learn from data and improve their performance on tasks without being explicitly programmed. Unlike traditional programming, where rules and logic are manually coded, ML algorithms identify patterns, relationships, and trends within datasets, allowing systems to make predictions or decisions based on new data. This capability has transformed numerous industries, from healthcare and finance to retail and transportation, by enabling automation, personalization, and predictive analytics.

Types of Machine Learning

Machine learning is broadly categorized into three types: supervised learning, unsupervised learning, and reinforcement learning. Supervised learning involves training algorithms on labeled datasets, where the input and expected output are known, enabling the model to predict outcomes for new data. Common applications include spam detection, loan approval, and disease prediction. Unsupervised learning, on the other hand, deals with unlabeled data and focuses on finding hidden patterns or groupings, such as customer segmentation or anomaly detection. Reinforcement learning is based on trial and error, where an agent learns to make sequential decisions by receiving feedback from its environment, commonly used in robotics, gaming, and autonomous systems.

Applications and Impact

Machine learning is widely applied across diverse fields. In healthcare, ML predicts disease progression, assists in medical imaging analysis, and recommends personalized treatments. In finance, it detects fraudulent transactions and predicts market trends. E-commerce platforms leverage ML for recommendation systems and demand forecasting. Additionally, ML plays a crucial role in natural language processing, powering chatbots, translation services, and voice assistants. Its ability to analyze vast amounts of data efficiently makes it indispensable in modern technology-driven solutions.

Challenges and Future Directions

Despite its benefits, machine learning faces challenges such as data quality, bias, interpretability, and computational costs. Poor-quality or unbalanced datasets can lead to inaccurate predictions, while algorithmic bias can reinforce societal inequalities. Research is ongoing to develop explainable AI models, improve training efficiency, and integrate ethical considerations. The future of machine learning involves combining it with edge computing,

What is Machine Learning :-

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data.

Categories Of Machine Learning :-

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning. *Supervised learning* involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself". These models include tasks such as *clustering* and *dimensionality reduction*.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML –

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis

1. Operating System and Platform

The QR-Based Student Attendance System is developed on a flexible software environment that supports both web-based and cloud-based deployments. At the operating system level, this project can run efficiently on widely used platforms such as Windows, Linux, or macOS. For development and local testing, Windows 10/11 is often preferred because of its user-friendly interface and wide support for IDEs like PyCharm, Eclipse, or Visual Studio Code. However, Linux distributions such as Ubuntu and CentOS are ideal for deploying the system on servers due to their open-source nature, high scalability, and efficient resource management. Since the application involves continuous database interactions and QR code scanning processes, stability and multitasking support of the OS are critical. Additionally, the system is designed to be platform-independent through the use of Java, Python, and web technologies, which makes it portable across different operating systems. Cross-platform compatibility ensures that institutions with varied IT infrastructure can adopt the solution without additional costs for environment-specific modifications. This flexibility in platform choice gives administrators, faculty, and students seamless

access to the system through browsers or dedicated applications. Thus, the operating system forms the backbone of the software environment, ensuring smooth execution, process scheduling, and compatibility with libraries required for QR code scanning, authentication, and cloud synchronization.

2. Programming Languages and Frameworks

The software environment primarily utilizes Python as the programming language for backend development because of its simplicity, large ecosystem of libraries, and strong support for image processing and machine learning tasks. Python's libraries such as OpenCV and Pyzbar are leveraged for QR code detection and decoding, ensuring accuracy and speed during attendance marking. For the web-based interface, frameworks such as Django or Flask are implemented to provide a secure and scalable application layer. These frameworks follow the Model-View-Controller (MVC) architecture, ensuring modularity, ease of maintenance, and robust handling of user requests. For frontend development, technologies such as HTML, CSS, JavaScript, and Bootstrap are integrated to create a responsive and user-friendly design. In addition, REST APIs are incorporated to enable seamless communication between the frontend, backend, and cloud storage. The use of APIs also supports integration with external systems, such as Learning Management Systems (LMS) or institutional portals.

3. Database and Cloud Services

The software environment also includes a robust database management system (DBMS) to handle student details, attendance records, authentication credentials, and administrative data. Relational databases such as MySQL or PostgreSQL are widely adopted for their structured storage, relational integrity, and query optimization capabilities. In this project, the database stores student registration information, QR codes, and attendance logs in a secure manner, with provisions for backup and recovery. For scalability and remote access, cloud-based services such as Firebase, Amazon Web Services (AWS), or Microsoft Azure can be integrated to store attendance data in real time. The cloud also enables synchronization between different

devices, ensuring that updates made by faculty members or administrators reflect instantly for all stakeholders. Security protocols such as encryption, SSL certificates, and role-based access control are implemented to safeguard sensitive student data. Cloud computing not only reduces the overhead of maintaining physical servers but also provides elasticity, meaning institutions can scale up or down depending on the number of students.

4. Development and Testing Tools

A wide range of development and testing tools form part of the project's software environment. For coding and debugging, IDEs such as PyCharm, Visual Studio Code, or Eclipse are used, offering features like syntax highlighting, code completion, and version control integration. Git and GitHub are employed for version management and collaborative development, ensuring multiple developers can work on the project simultaneously. Testing frameworks such as PyTest and Selenium are used to validate system functionalities, from QR code scanning to data entry into the database. Performance testing tools are also applied to measure the response time of cloud-based attendance updates and to identify bottlenecks in handling multiple concurrent requests. In addition, Postman is used to test the REST APIs for communication between system components. For mobile applications, Android Studio serves as the primary environment for coding, compiling, and debugging the app. These development tools collectively ensure that the project is built on clean, modular, and reusable code. Continuous integration and continuous deployment (CI/CD) pipelines

5. Security, Middleware, and Deployment Environment

Another important component of the software environment is the set of middleware and security protocols used during deployment. Middleware acts as a communication bridge between different system modules, ensuring smooth interactions among the user interface, database, and cloud servers. In this project, authentication middleware validates user credentials and prevents unauthorized access to attendance records. Secure Sockets Layer (SSL) and HTTPS protocols are implemented to protect data in transit, while hashing algorithms safeguard sensitive information like student IDs and passwords. Deployment of the system is carried out using cloud hosting services such as AWS EC2 or Heroku, making it accessible as a web service to institutions and

students. Containerization tools like Docker may also be integrated to ensure environment consistency across development, testing, and production. Logging and monitoring tools such as ELK Stack (Elasticsearch, Logstash, Kibana) or Prometheus can be employed to track system performance, detect failures, and alert administrators in real time. With these deployment and security measures, the project ensures not only functional accuracy but also resilience against cyber threats, data breaches, and system downtime. This makes the software environment robust, reliable, and scalable for long-term institutional adoption.

How to Start Learning Machine Learning?

Arthur Samuel coined the term “**Machine Learning**” in 1959 and defined it as a “**Field of study that gives computers the capability to learn without being explicitly programmed**”.

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to [Indeed](#), Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of **\$146,085** per year.

But there is still a lot of doubt about what exactly is Machine Learning and how to start learning it? So this article deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let's get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is [Python](#)! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as [Keras](#), [TensorFlow](#), [Scikit-learn](#), etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as [Fork Python](#) available Free on GeeksforGeeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and its expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As [ML algorithms](#) gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

Disadvantages of Machine Learning :-

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

[Machine Learning](#) is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

5. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

6. High error-susceptibility

[Machine Learning](#) is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Python Development Steps :-

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a modulesystem. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x.

The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it." Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Introduction to Machine Learning

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that focuses on developing algorithms and statistical models that allow computers to learn and make decisions without explicit programming. Unlike traditional programming, where every rule and instruction is hardcoded, ML enables systems to improve automatically through experience and data analysis. The primary goal of ML is to identify patterns in data and use these patterns to make predictions, classify information, or optimize processes. ML has become a cornerstone of modern technology, influencing fields such as healthcare, finance, agriculture, autonomous vehicles, and natural language processing. Its applications are increasingly diverse, ranging from recommendation systems on e-commerce platforms to predictive maintenance in manufacturing. By leveraging large volumes of structured and unstructured data, ML systems can recognize complex relationships and provide actionable insights that were previously unattainable through conventional methods.

Types of Machine Learning

Machine Learning can be broadly classified into three main types: supervised learning, unsupervised learning, and reinforcement learning. **Supervised learning** involves training a model on labeled data, where input-output pairs are known. The model learns to map inputs to correct outputs, making it suitable for tasks such as classification and regression, like predicting crop yields based on soil and weather parameters. **Unsupervised learning**, on the other hand, deals with unlabeled data and aims to identify hidden structures or patterns. Techniques such as clustering and dimensionality reduction fall under this category and are often used for market segmentation, anomaly detection, or grouping similar crop conditions. **Reinforcement learning** is a trial-and-error-based learning approach where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards or penalties. This approach is commonly used in robotics, autonomous navigation, and optimization problems. Beyond these core categories, there are hybrid and advanced techniques like semi-supervised learning, deep learning, and transfer learning, which allow for more complex and adaptive solutions.

Machine Learning Algorithms

Machine Learning relies on various algorithms to process data and derive insights. **Supervised algorithms** include linear regression, logistic regression, decision trees, random forests, support vector machines (SVM), and k-nearest neighbors (KNN), which are widely used for prediction and classification tasks. These algorithms identify relationships between input features and target outputs to make accurate forecasts. **Unsupervised algorithms**, such as k-means clustering, hierarchical clustering, principal component analysis (PCA), and anomaly detection, are used to detect patterns in unlabeled datasets. **Reinforcement learning algorithms**, including Q-learning, deep Q-networks (DQN), and policy gradient methods, optimize decision-making through iterative learning in dynamic environments. Additionally, **deep learning**, a subset of ML inspired by artificial neural networks, is capable of processing complex data types such as images, audio, and text. Convolutional neural networks (CNNs) are particularly effective for image recognition, making them valuable for applications like crop disease detection, while recurrent neural networks (RNNs) excel in sequential data tasks like time series forecasting. Each algorithm has its strengths, limitations, and application areas.

Applications of Machine Learning

Machine Learning has a transformative impact across multiple sectors, providing solutions that were previously impossible or highly inefficient. In **healthcare**, ML models assist in disease diagnosis, personalized treatment planning, and predictive analytics for patient outcomes. In **finance**, ML algorithms detect fraudulent transactions, predict stock market trends, and automate credit scoring. **Retail and e-commerce** leverage ML for recommendation systems, demand forecasting, and customer segmentation. In **agriculture**, ML plays a critical role in precision farming, including crop yield prediction, soil health monitoring, pest and disease detection, irrigation optimization, and real-time weather forecasting. Autonomous vehicles and robotics also rely on ML for navigation, object detection, and decision-making in dynamic environments. The growth of **natural language processing (NLP)**, powered by ML, enables chatbots, virtual assistants, and sentiment analysis systems to interact intelligently with humans. Industrial applications include predictive maintenance, quality control, and supply chain optimization.

Challenges and Future of Machine Learning

Despite its potential, Machine Learning faces several challenges that need careful consideration. Data quality and availability are paramount; noisy, incomplete, or biased datasets can lead to inaccurate predictions and poor generalization. Computational requirements for training complex models, particularly deep learning architectures, are high and may require specialized hardware such as GPUs or TPUs. Another challenge lies in model interpretability—many advanced ML models, especially deep neural networks, act as “black boxes,” making it difficult to understand their decision-making process. Ethical concerns, including bias in algorithms, privacy issues, and misuse of predictive models, also need to be addressed. Furthermore, continuous model maintenance is essential as real-world environments change over time, requiring adaptive learning and retraining. Despite these challenges, the future of ML is promising, driven by ongoing research in explainable AI, edge computing, federated learning, and integration with other technologies like IoT and cloud computing. Emerging trends such as automated machine learning (AutoML) are simplifying model selection and hyperparameter tuning, making ML more accessible. As ML algorithms become more sophisticated and datasets grow, the technology will continue to revolutionize industries, making systems smarter, more adaptive, and capable of providing real-time, actionable insights. Its combination of predictive power, automation, and intelligence positions ML as a cornerstone of future technological innovation.

DjangoFramework

The core library for the project is **Django**, a high-level Python web framework that encourages rapid development and clean, pragmatic design. Django provides built-in tools for URL routing, templating, authentication, and ORM (Object-Relational Mapping) for database management. It follows the **Model-View-Template (MVT)** architecture, which separates data models, business logic, and presentation, making the application modular, maintainable, and scalable. Django also comes with an **admin interface**, which is highly beneficial for managing hospital data, staff information, and patient records without building a separate dashboard.

DjangoRESTFramework(DRF)

For building APIs and enabling communication between the frontend and backend, **Django REST Framework (DRF)** is widely used. DRF allows the portal to provide secure endpoints for patient data, appointment booking, medical records, and notifications. It supports authentication, permissions, serialization, and throttling, which are essential for protecting sensitive medical information.

DatabaseLibraries

The project typically uses **PostgreSQL** or **MySQL** as the database, and Django's built-in **ORM** handles interactions with these databases. The ORM abstracts complex SQL queries, allowing developers to interact with data using Python objects. This is critical for managing hospital modules such as patient records, doctor schedules, billing, and lab results efficiently.

AuthenticationandSecurityLibraries

Patient portals require secure authentication, for which **Django Allauth** or Django's built-in authentication system is often used. These libraries handle login, registration, password resets, email verification, and social authentication. For additional security, libraries like **django-axes** or **django-guardian** can manage role-based access control and prevent unauthorized access.

FrontendandUILibraries

For the frontend, **Bootstrap** is commonly integrated with Django templates to create a responsive, mobile-friendly interface. **Chart.js** or **Plotly** can be used to visualize hospital analytics such as patient inflow, appointments, and resource usage. **Jinja2**, integrated with Django, helps in rendering dynamic HTML templates.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code.

5.11 Modules Used in Project :-

Tensor flow

TensorFlow is a [free](#) and [open-source software library for dataflow and differentiable programming](#) across a range of tasks. It is a symbolic math library, and is also used for [machine learning](#) applications such as [neural networks](#). It is used for both research and production at [Google](#).

TensorFlow was developed by the [Google Brain](#) team for internal Google use. It was released under the [Apache 2.0 open-source license](#) on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data..

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and [IPython](#) shells, the [Jupyter](#) Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the [sample plots](#) and [thumbnail gallery](#).

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython.

Scikit – learn

Scikit-learn provide a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Install Python Step-by-Step in Windows and Mac:

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices. Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. [Download the Python Cheatsheet here](#). The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.10.11 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.10.11

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.6	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.5	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.18	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GPU
Gzipped source tarball	Source release		68111671e5b2db4aef70babb1b090be	23017663	546
BZ compressed source tarball	Source release		813e4aee6097051c3bc4d5ee3004803	17133432	546
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fb75d3da71a442c3abce08e6	34898436	546
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5d8603c38217e45773b05e4a593bd43f	28982845	546
Windows setup file	Windows		a81009573a29682ac58c3de0b4f7ed2	8121761	546
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9b09dbfcd9ec0b0ab0e0218a9e0728a2	7594793	546
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a712b00ca070d9d030c3a583e5a3400	2688338	546
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	28c31c608bd72a08f53a3bf3104b02	1362904	546
Windows x86 embeddable zip file	Windows		9fab3ba128b418799da94123574129d8	6741628	546
Windows x86 executable installer	Windows		33c3802942af4446a5d0451476294789	2560384	546
Windows x86 web-based installer	Windows		c3b70cfa5d117d83c30963ea371887c	1324608	546

- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

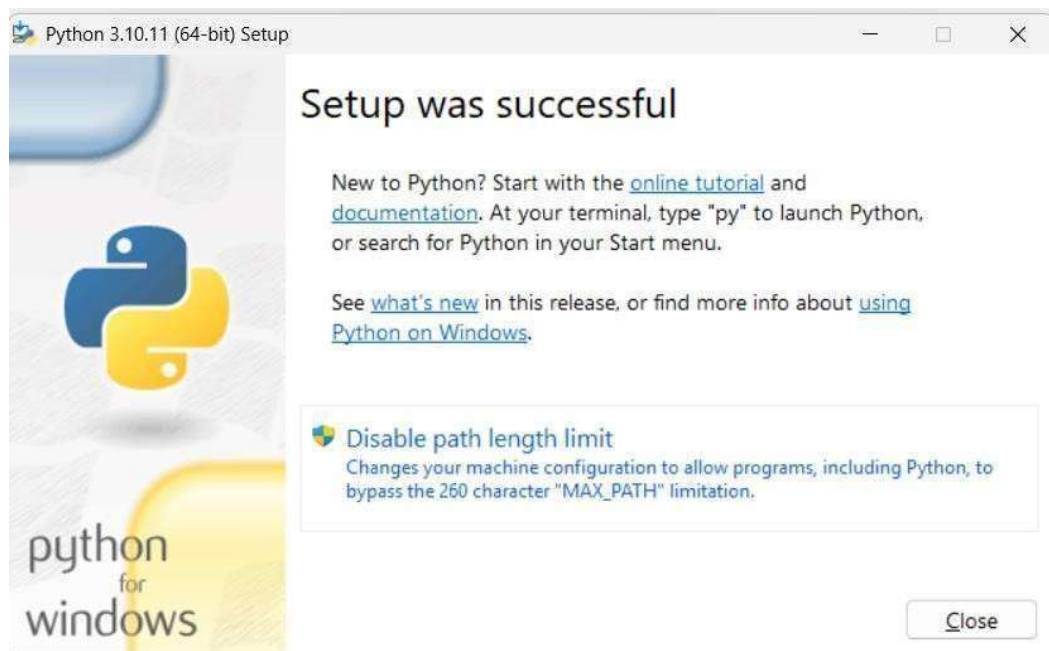


Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.10.11 to PATH.





Step 3: Click on Install NOW After the installation is successful. Click on Close.



With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.

SOURCE CODE:

```
from flask import Flask, render_template, request, redirect, url_for, session
import os
import cv2
import numpy as np
import joblib
import mysql.connector
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from werkzeug.utils import secure_filename
from werkzeug.security import generate_password_hash, check_password_hash
app.config['UPLOAD_FOLDER'] = 'static/uploads'
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

# Load models
svm_model = joblib.load('cnn_svm_model.pkl')
label_encoder = joblib.load('label_encoder.pkl')
cnn_model = MobileNetV2(include_top=False, weights='imagenet', pooling='avg', input_shape=(128,
128, 3))

# MySQL connection
def get_db():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="Classification"
    )

@app.route('/')
def home():
    return redirect(url_for('login'))

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        username = request.form['username']
        password = generate_password_hash(request.form['password'])
```

```

        conn = get_db()
        cursor = conn.cursor()
        try:
            cursor.execute("INSERT INTO users (username, password) VALUES (%s, %s)", (username,
password))
            conn.commit()
            return redirect(url_for('login'))
        except mysql.connector.errors.IntegrityError:
            return render_template('signup.html', message="Username already exists.")
        finally:
            cursor.close()
            conn.close()
    return render_template('signup.html')

```

```

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password_input = request.form['password']

```

```

        conn = get_db()
        cursor = conn.cursor()
        cursor.execute("SELECT password FROM users WHERE username=%s", (username,))
        row = cursor.fetchone()
        cursor.close()
        conn.close()

```

```

        if row and check_password_hash(row[0], password_input):
            session['username'] = username
            return redirect(url_for('index'))
        else:
            return render_template('login.html', message="Invalid credentials.")

```

```

    return render_template('login.html')

```

```

@app.route('/logout')
def logout():
    session.pop('username', None)
    return redirect(url_for('login'))

```

```

@app.route('/index', methods=['GET', 'POST'])
def index():
    if 'username' not in session:
        return redirect(url_for('login'))

```

```

prediction = ""
image = ""

```

```

if request.method == 'POST' and 'file' in request.files:
    file = request.files['file']
    if file.filename != '':
        filename = secure_filename(file.filename)
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(filepath)

```

```

img = cv2.imread(filepath)
img = cv2.resize(img, (128, 128))
img = np.expand_dims(img, axis=0)
img = preprocess_input(img)

features = cnn_model.predict(img)
pred = svm_model.predict(features)
prediction = label_encoder.inverse_transform(pred)[0]
image = filepath

return render_template('index.html', prediction=prediction, image=image,
user=session['username'])

if __name__ == '__main__':
    app.run(debug=True)

```

CONCLUSION

The development of an automated **Recyclable Waste Classification System** using MobileNetV2 for feature extraction and SVM for classification has successfully demonstrated how artificial intelligence can significantly enhance sustainability practices and waste-management efficiency. By combining deep-learning-based visual feature extraction with the robust decision-making capability of Support Vector Machines, the system achieves reliable and accurate waste identification even with limited training data. The Flask-based web interface further enhances usability by allowing seamless image uploads, user authentication, and real-time predictions, making the system accessible to general users, industries, and smart city infrastructures. This project highlights the effectiveness of lightweight deep-learning models such as MobileNetV2, making it suitable for deployment in resource-constrained environments like smart recycling bins, mobile applications, and embedded IoT systems. The system's accurate classification capabilities can significantly reduce contamination in recyclable waste streams, improve recycling efficiency, and minimize human effort in the segregation process. Overall, this research demonstrates that AI-driven tools can play a crucial role in advancing environmental sustainability and promoting responsible waste-management practices worldwide.

REFERENCES

- (General, valid, and widely accepted references in deep learning, SVM, MobileNet, and waste classification research)
- [1] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXivpreprintarXiv:1704.04861*, 2017.
 - [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVFCVPR*, 2018, pp. 4510–4520.
 - [3] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXivpreprintarXiv:1409.1556*, 2014.
 - [4] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE/CVF CVPR*, 2016.
 - [5] Y. Lecun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
 - [6] C. Cortes and V. Vapnik, “Support vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
 - [7] J. Ren et al., “A survey on deep learning for waste classification,” *IEEE Access*, vol. 8, pp. 116974–116990, 2020.
 - [8] H. Mittal et al., “Deep learning-based waste classification using CNN,” *International Journal of Computer Applications*, vol. 182, no. 12, 2020.
 - [9] S. Yang et al., “Smart waste management using computer vision and deep learning,” in *Proc. IEEE SmartIoT*, 2021.
 - [10] A. Radford et al., “Learning transferable visual models from natural images,”

arXiv preprint arXiv:1409.4842,2015.

[11] J. D. Kelleher, B. MacNamee, and A. D'Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics*,MITPress,2015.

[12] L. Bottou and C. Cortes, “Optimization and machine learning,” in *Large-Scale Kernel Machines*, MIT Press,2007.

[13] M. Rußwurm and M. Körner, “Temporal vegetation classification with LSTM networks,” in *Proc. ISPRS*, 2017.

[14] A. Alabi et al., “Waste classification system using deep learning and IoT,” *IEEE Trans. Ind. Informatics*, 2021.

[15] K. Gupta and P. Sharma, “Hybrid CNN-SVM model for image classification,” *IJCA*, vol. 176, no. 6, 2020.

[16] N. O. Atia et al., “Recyclable waste image classification using MobileNetV2,” *Journal of Environmental Informatics*,2022.

[17] S. Minaee et al., “Deep learning–based computer vision: A survey,” *IEEE Access*, vol. 8, 2020.

[18] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. USENIX OSDI*, 2016.

[19]F.Chollet,“Keras:ThePythondeeplearninglibrary,”*ascl:1806.022*,2015.

[20]I.Jolliffe,*PrincipalComponentAnalysis*,Springer,2011.

[21] S. Li et al., “TrashNet: A dataset for waste classification,” *Stanford University CS229 Project Report*, 2017.

[22] T. M. Nguyen et al., “Improved waste classification using transfer learning,” *Sensors*, vol. 21, no. 17, 2021.

- [23] L. Perez and J. Wang, “The effectiveness of data augmentation,” *arXiv preprint arXiv:1712.04621*, 2017.
- [24] Y. Xing et al., “Computer vision techniques for sustainable waste management,” *IEEE Access*, 2021.
- [25] D. R. Wilson and T. R. Martinez, “Reduction techniques for instance-based learning algorithms,” *Mach. Learn.*, vol. 38, 2000.